JALA ACADEMY RESTful API ASSIGNMENT

API Test Cases, Test Scenarios, Expected Results & cURL Examples

1. Objective

The objective of this assignment is to test the JALA Academy RESTful User APIs using Postman or cURL. The testing covers authentication, user creation, input validation, user update, user search, and user deletion. Positive and negative scenarios are included to verify that the API accepts valid data and handles invalid data safely.

2. API Environment and Authentication

Item	Details
Base URL	http://trainingapi.jalaacademy.com/api
Login endpoint	POST /signin
User endpoints	POST /users, PUT /users, GET /users, DELETE /users (use the exact path/query format shown in the supplied Postman collection).
Request format	Accept: application/json; request body: JSON where applicable.
Authentication	First call POST /signin with valid credentials. Copy the returned bearer token and add it to the Authorization tab as Bearer Token for all user APIs.
Login test data	Valid: admin@jalaacademy.com / admin123. Invalid: admin@admin.com / admin123.

Important: The supplied API description does not show the response body, exact update payload, or search/delete parameter names. For those requests, use the exact URL, parameter names, and JSON structure displayed in the JALA Postman collection. Do not assume a status code or response message that is not returned by the live API.

3. Login API Test Cases

TC ID	Scenario / Request	Test Data	Expected Result
AUTH-01	POST /signin – valid credentials	admin@jalaacademy.com / admin123	Authentication succeeds and a bearer token is generated/returned. Token can be copied for subsequent API calls.
AUTH-02	POST /signin – invalid credentials	admin@admin.com / admin123	Authentication is rejected. No valid bearer token should be issued; an appropriate error response should be returned.

4. User Creation API – POST /users

Sample valid JSON request body

```
{
  "first_name": "Test",
  "last_name": "User",
  "email": "test1@teset.com",
  "mobile": "1004567822",
  "dob": "1995-03-06",
  "gender": "male",
  "address": "Test address",
  "country": "india",
  "skills": ["aws", "fullstack"]
}
```

TC ID	Scenario	Test Input	Expected Result
CREATE-01	CreateUser_ValidInput	All fields contain valid values; use a unique email/mobile.	User is created successfully and the API returns a success response with the created user details or identifier.
CREATE-02	CreateUser_InvalidInput	Send invalid field values or incorrect data types.	Request is rejected with validation errors; invalid user data is not saved.
CREATE-03	CreateUser_EmptyFirstName	Set first_name to an empty string.	Request is rejected and first_name validation is reported.
CREATE-04	CreateUser_EmptyLastName	Set last_name to an empty string.	Request is rejected and last_name validation is reported.

TC ID	Scenario	Test Input	Expected Result
CREATE-05	CreateUser_ValidEmail	Use a valid email format, e.g. sathwika.test@example.com.	Email format is accepted, provided it is not already registered.
CREATE-06	CreateUser_InvalidEmail	Use an invalid email, e.g. sathwika.test@.	Request is rejected with an email validation error.
CREATE-07	CreateUser_ValidMobile	Use a valid mobile number in the format/range required by the API.	Mobile number is accepted if it is not already registered.
CREATE-08	CreateUser_InvalidMobile	Use alphabetic characters, too few digits, or an invalid mobile format.	Request is rejected with a mobile validation error.

5. User Update API – PUT

TC ID	Scenario	Test Input	Expected Result
UPDATE-01	UpdateUser_ValidInput	Update an existing user with valid first name, last name, email, mobile, and other supported fields.	User details are updated successfully; a subsequent search shows the updated values.
UPDATE-02	UpdateUser_InvalidInput	Send invalid field values or an invalid user identifier.	Request is rejected; existing user data remains unchanged.
UPDATE-03	UpdateUser_EmptyFirstName	Set first_name to empty.	Validation error is returned; first name is not overwritten with an empty value.
UPDATE-04	UpdateUser_EmptyLastName	Set last_name to empty.	Validation error is returned; last name is not overwritten with an empty value.
UPDATE-05	UpdateUser_ValidEmail	Set email to a valid, unused email address.	Email is accepted and updated.
UPDATE-06	UpdateUser_InvalidEmail	Set email to an invalid format such as user@.	Validation error is returned; invalid email is not saved.
UPDATE-07	UpdateUser_ValidMobile	Set mobile to a valid, unused mobile number.	Mobile is accepted and updated.
UPDATE-08	UpdateUser_InvalidMobile	Set mobile to letters, too few digits, or an unsupported format.	Validation error is returned; invalid mobile is not saved.

Update request note

Use the update URL and user identifier format specified in the supplied JALA collection. Send the supported user fields in raw JSON and include the bearer token. After updating, run SearchUser_ValidInput to verify persistence.

6. User Search API – GET

TC ID	Scenario	Test Input	Expected Result
SEARCH-01	SearchUser_ValidInput	Search using an existing user ID/email/mobile or the valid search parameter specified in the collection.	Matching user record is returned with correct details.
SEARCH-02	SearchUser_InvalidInput	Search using a non-existing identifier, malformed value, or invalid parameter.	No unrelated user is returned; API responds with an appropriate not-found or validation result.

7. User Delete API – DELETE

TC ID	Scenario	Test Input	Expected Result
DELETE-01	DeleteUser_ValidID	Delete an existing user using a valid ID.	User is deleted successfully. A subsequent search for the same ID should not return the deleted record.
DELETE-02	DeleteUser_InvalidID	Use a non-existing or malformed user ID.	Delete operation is rejected or reports that the user was not found; no other user is deleted.
DELETE-03	DeleteUser_ValidMobileNo	Delete an existing user using that user's registered mobile number.	The matching user is deleted successfully; subsequent search confirms deletion.
DELETE-04	DeleteUser_InvalidMobileNo	Use an unregistered or invalid mobile number.	API reports no matching user or validation failure; no user is deleted.
DELETE-05	DeleteUser_ValidEmail	Delete an existing user using that user's registered email.	The matching user is deleted successfully; subsequent search confirms deletion.
DELETE-06	DeleteUser_InvalidEmail	Use an unregistered or malformed email address.	API reports no matching user or validation failure; no user is deleted.

8. Sample cURL Commands

A. Login – valid input

```
curl --location 'http://trainingapi.jalaacademy.com/api/signin' \\ --header 'Accept: application/json' \\
--header 'Content-Type: application/json' \\ --data-raw
'{"email":"admin@jalaacademy.com","password":"admin123"}'
```

B. Create user – valid input (replace the sample email/mobile with unique values)

```
curl --location 'http://trainingapi.jalaacademy.com/api/users' \\ --request POST \\ --header 'Accept:
application/json' \\ --header 'Content-Type: application/json' \\ --header 'Authorization: Bearer ' \\
```

```
--data-raw '{"first_name":"Test","last_name":"User","email":"test1@example.com","mobile":"1004567822","dob":
"1995-03-06","gender":"male","address":"Test address","country":"india","skills":["aws","fullstack"]}'
```

C. Update, search, and delete

Use the exact endpoint URL and request/query parameters provided for each request in the JALA Postman collection. Add Authorization: Bearer and Accept: application/json. Select PUT for update, GET for search, and DELETE for deletion. Do not reuse the same user record for multiple destructive tests; create separate test records where needed.

9. Execution Procedure in Postman

1. Open the JALA Academy RESTful API collection in Postman.
2. Run Login_ValidInput using the supplied valid credentials.
3. Copy the bearer token from the login response/authorization details.
4. Open each user API request and set Authorization type to Bearer Token; paste the token.
5. Set Accept: application/json. For JSON request bodies, choose Body → raw → JSON.
6. Execute the positive and negative test cases listed in this document.
7. Record the actual HTTP status, response body, and result for every test case.
8. After create/update/delete operations, use search to verify the resulting data.
9. Mark each test case Pass only when the actual result matches the expected result; otherwise mark Fail and attach evidence.

10. General Validation Checklist

• Valid login generates a token; invalid login does not generate a usable token.

• Protected APIs reject requests when the token is missing, invalid, or expired.

• Required fields such as first_name and last_name cannot be blank.

• Email and mobile fields accept valid formats and reject invalid formats.

• Duplicate email/mobile values are handled according to the API's validation rules.

• Created and updated values are persisted and can be verified through search.

• Invalid identifiers do not update or delete unrelated users.

• Deleted users are no longer returned by search.

• API responses use JSON where applicable and provide meaningful success/error information.

11. Conclusion

The JALA Academy RESTful API test plan covers 26 scenarios across authentication and user management: 2 login, 8 create, 8 update, 2 search, and 6 delete scenarios. These tests help verify API functionality, input validation, bearer-token security, and data integrity. Actual status codes and response messages should be recorded from the live API during execution.